

SILKPAY OPENAI 商户接入手册

本页面由 Markdown 源文档自动渲染，用于对外商户接入交付与 PDF 导出。

版本：v1.4 更新时间：2026-05-18 适用对象：已开通 Speed Pay OpenAPI 的商户技术团队 当前覆盖范围：收款、退款、争议、可用代收渠道查询、Webhook、沙箱联调

1. 文档定位

本文面向商户技术接入，不讨论平台后台、代理后台或上游通道管理，

当前 OpenAPI 只暴露商户实际联调需要的最小接口集：

- 创建收款
- 查询收款
- 查询当前可用代收渠道
- 创建退款
- 批量查询退款
- 查询争议
- Webhook 事件通知

平台仍控制上游通道启停、默认路由和商户可见能力边界，

当前商户 OpenAPI 只暴露“可联调、可使用”的代收渠道列表，不暴露平台侧通道管理能力。

2. 接入前准备

平台为每个商户分配以下接入资料：

项目	用途
<code>merchant_id</code>	OpenAPI Basic Auth 用户名
<code>merchant_secret_key</code>	OpenAPI Basic Auth 密码
<code>merchant_webhook_secret</code>	Webhook 验签密钥，原始值为 Base64 编码字符串
<code>environment</code>	商户环境，取值为 <code>LIVE</code> 或 <code>SANDBOX</code>

平台还会为商户配置一组“集成策略”，不一定全部开放给商户自行修改：

项目	用途
<code>notify_url</code>	商户默认 Webhook 回调地址；当单笔订单未单独指定 <code>notify_url</code> 时使用
<code>return_url</code>	商户默认支付完成跳转地址；当单笔订单未单独指定 <code>redirect_url</code> 时使用
<code>openapi_ip_allowlist</code>	OpenAPI 请求来源 IP 白名单，支持单 IP 与 CIDR；为空表示不限制
<code>callback_host_allowlist</code>	商户回调域名白名单；为空时兼容旧行为，非空时限制 <code>notify_url</code> / <code>return_url</code> 可使用的 hostname
<code>allow_notify_url_override</code>	是否允许在单笔 <code>charge.create</code> 中传 <code>charge.extra.notify_url</code> 覆盖默认回调地址
<code>allow_callback_profile_self_manage</code>	是否允许商户在商户后台自行修改默认 <code>notify_url</code> / <code>return_url</code>

说明：

- `merchant_id` 表示平台分配给你的商户号，也是平台内部商户编码。

- `environment=SANDBOX` 表示该商户只用于联调，流程会完整执行，但不会调用真实上游，也不会产生真实资金流。
- `environment=LIVE` 表示真实生产商户，会按平台路由调用真实上游。
- 默认 `notify_url` 或 `return_url` 可以由平台在创建商户时配置，也可以在后续由平台修改。
- 若平台配置了 `callback_host_allowlist`，则默认 `notify_url` / `return_url` 与单笔请求级覆盖地址都必须命中白名单 `hostname`。
- 若平台未开启 `allow_callback_profile_self_manage`，商户侧不能自行修改默认回调地址。
- 平台可能允许或禁止你在下单时自定义代收渠道；是否开通以平台配置为准。
- 建议联调和正式生产分别使用不同商户号，不要把同一个商户号在联调和生产之间来回切换，

接入前建议按以下顺序完成：

1. 先用 `SANDBOX` 商户打通 `charge.create` -> `webhook` -> `charge.retrieve`。
2. 再接入 `refund.create` -> `refund.batch-retrieve`。
3. 最后接入 `dispute.retrieve` 与争议处理流程。

3. 接入模式与域名

当前 Silk pay 采用“单套部署 + 商户级环境”模式，不再为商户提供单独的 `api-sbx` 域名。

项目	说明
统一 OpenAPI 域名	<code>https://api.speed-pay.nsngc.org</code>
<code>SANDBOX</code> 商户	仍走完整 OpenAPI / Webhook / 回放链路，但不调用真实上游
<code>LIVE</code> 商户	调用真实上游，产生真实资金与结算动作

也就是说，联调与生产的差异由商户本身的 `environment` 决定，而不是靠切换另一套 API 域名。

4. 认证方式

所有 OpenAPI 均要求：

- 使用 HTTPS
- 使用 POST
- 请求体为 application/json
- 使用 HTTP Basic Auth

Basic Auth 规则：

```
username = merchant_id
password = merchant_secret_key
Authorization = "Basic " + Base64(merchant_id + ":" +
merchant_secret_key)
```

curl 示例：

```
curl -X POST "https://api.speed-
pay.nsngc.org/ganymede/open/v1/charge/retrieve" \
-u "$MERCHANT_ID:$MERCHANT_SECRET_KEY" \
-H "Content-Type: application/json" \
-d '{"charge_id":"ch_demo_001"}'
```

5. 公共请求约定

5.1 响应包结构

业务成功或失败均遵循统一 JSON 包结构：

```
{
  "code": 1,
  "message": "ok",
  "data": {}
}
```

处理时要区分两层状态：

- HTTP 层：传输、认证、并发幂等冲突
- 业务层：code、message、对象状态字段

5.2 HTTP 层语义

HTTP 状态码	说明
200	请求已被系统接收，结果看响应体 code 与业务对象状态
401	Basic Auth 失败
403	请求来源 IP 不在 OpenAPI 白名单内
409	同一个幂等键存在并发请求，前一个请求尚未处理完成

5.3 金额与币种

- 所有金额都使用**整数最小货币单位**，不能传小数或浮点数。
- 例如 USD 100 表示 1.00 USD；USD 71.99 应传 7199。
- `charge.charge_amount` 与 `charge.merchant_order.order_items[].item_unit_amount` 都遵循该规则。
- 不要用浮点数直接计算金额后提交；例如 JavaScript 中 `71.99 * 100` 可能得到 7198.999999999999，该值不是整数，会被判定为请求数据格式错误。
- 商品明细金额应与收款金额保持一致：`sum(item_unit_amount * quantity) == charge_amount`。
- 币种使用三位大写 ISO 代码，如 USD、EUR。

5.4 时间字段

- 所有时间戳均为秒级 UNIX 时间戳。
- 例如 1752198585。

5.5 幂等性

以下写接口支持幂等：

- POST /ganymede/open/v1/charge/create
- POST /ganymede/open/v1/refund/create

幂等请求头：

X-BF-IdempotencyKey: <your-unique-key>

约定如下：

- 幂等保障窗口不少于 12 小时。
- 同一幂等键重复请求，会返回第一次请求的同一结果。
- 同一幂等键并发请求时，可能返回 HTTP 409 。
- 建议直接使用业务唯一号，如订单号、退款号。

5.6 metadata

charge.metadata 与 refund.metadata 都可承载商户自定义附加信息，但当前协议形态不同：

- charge.metadata 使用 JSON 对象
- refund.metadata 当前使用字符串；如需传结构化数据，建议传 JSON 字符串

约束如下：

- 只支持单层结构
- 总长度不超过 1024 byte
- 建议只放对账、归因、风控辅助字段

示例：

```
{
  "order_no": "S0202603240001",
  "utm_source": "merchant.example"
}
```

退款示例值：


```
"{\\"refund_no\\":\\"RF202603240001\\"}"
```

5.7 商户号返回规则

所有响应与 Webhook 中面向商户侧展示的 `merchant_id`，都表示你在 Speed Pay 平台上的商户号，不会暴露上游通道分配的商户号。

5.8 OpenAPI IP 白名单

平台可为商户配置 `openapi_ip_allowlist`，用于限制 OpenAPI 请求来源：

- 白名单校验发生在 Basic Auth 通过之后。
- 支持单 IP，例如 `203.0.113.10`。
- 支持 CIDR，例如 `203.0.113.0/24`。
- 白名单为空时表示不限制来源 IP。
- 当请求来源 IP 不在白名单内时，接口直接返回 HTTP 403。
- 商户侧不能通过 OpenAPI 自行修改该白名单，需由平台侧配置。

5.9 回调地址策略

Speed Pay 同时区分“默认回调配置”和“单笔请求级覆盖”：

- `notify_url`：服务端异步 Webhook 投递地址。
- `return_url` / `redirect_url`：用户支付完成后的前端跳转地址，不参与服务端 Webhook 签名或重试。

最终地址决策规则如下：

- 最终 `notify_url` 优先级：`charge.extra.notify_url`（仅平台开启 `allow_notify_url_override` 时）-> 商户默认 `notify_url` -> 空。
- 最终 `return_url` 优先级：`charge.extra.redirect_url` -> 商户默认 `return_url` -> 空。
- 若 `charge.extra.enable_charge_link=true` 且最终 `return_url` 为空，接口返回错误码 1016。
- 若最终 `notify_url` 为空，系统仍会完成订单状态流转与事件落库，但不会创建 callback delivery，也不会发起重试。

当平台为商户配置 `callback_host_allowlist` 时，还会额外应用以下规则：

- 白名单为空时，回调地址策略保持旧行为，不额外限制 `hostname`。
- 白名单非空时，默认 `notify_url` / `return_url` 与请求级 `charge.extra.notify_url` / `charge.extra.redirect_url` 都必须命中白名单。
- 白名单只做精确 `hostname` 匹配；`scheme`、`path`、`query`、`fragment` 与 `port` 不参与匹配。
- 不支持 wildcard，例如 `*.merchant.example` 不会生效；子域名也不会自动放行。
- 任一地址标准化后未提取到合法 `hostname`，或 `hostname` 不在白名单内，请求会按参数校验失败拒绝。

当平台为商户开启“平台托管支付结果页”时，还会额外应用以下规则：

- 商户传入的原始 `redirect_url` 不会丢失，平台会将其保存为“外部返回地址”。
- 商户原始 `redirect_url` 会先按上述 `callback_host_allowlist` 规则校验；校验通过后，真正发给上游的 `redirect_url` 才会被改写为平台结果页地址，例如 `https://api.speed-pay.nsngc.org/pay/result?t=...`。
- 平台结果页域名属于平台内部托管域名，不需要加入商户自己的 `callback_host_allowlist`。
- 用户从上游返回后，平台结果页会展示当前已确认状态，并在有外部返回地址时提供“返回商户站点”按钮。
- 该结果页只是浏览器回跳承接页，不替代 Webhook；商户仍应以 Webhook 或后端补查作为最终到账依据。

地址校验规则如下：

- LIVE 商户：`notify_url` / `return_url` 必须使用 `https`。
- SANDBOX 商户：`notify_url` / `return_url` 允许 `http` 或 `https`。
- 所有环境均禁止 `localhost`、内网地址、保留地址或其他不可公开访问地址。

6. 标准接入流程

建议把 Webhook 作为最终状态源，主动查询作为补偿和对账手段。

6.1 收款主流程

1. 商户服务端调用 `charge.create` 创建收款。
2. 前端根据返回的 `credential` 拉起支付。
3. 用户完成支付或 3DS 验证。
4. 平台异步推送 `charge.succeeded` 或 `charge.failed` 到 `notify_url`。
5. 商户收到 Webhook 后更新订单状态。
6. 如用户前端先返回但 Webhook 未到，可调用 `charge.retrieve` 做补查。

6.2 退款流程

1. 商户服务端调用 `refund.create` 发起退款。
2. 平台异步推送 `refund.succeeded` 或 `refund.failed`。
3. 如需批量对账，可调用 `refund.batch-retrieve`。

6.3 争议流程

1. 平台异步推送争议事件。
2. 商户根据 `dispute_id` 调用 `dispute.retrieve` 拉取最新争议详情。

7. 接口清单

接口	方法	路径	是否幂等	说明
创建收款	POST	/ganymede/open/v1/charge/create	是	创建支付单
查询收款	POST	/ganymede/open/v1/charge/retrieve	否	查询支付状态
查询可用代收渠道	GET	/ganymede/open/v1/channel/collection-list	否	返回当前商户可见的代收渠道列表
创建退款	POST	/ganymede/open/v1/refund/create	是	对已存在支付单发起退款
批量查询退款	POST	/ganymede/open/v1/refund/batch-retrieve	否	按退款号批量查状态
查询争议	POST	/ganymede/open/v1/dispute/retrieve	否	查询单笔争议详情

8. 查询可用代收渠道

8.1 请求路径

GET /ganymede/open/v1/channel/collection-list

8.2 使用场景

- 商户后台让运营查看当前可用代收渠道。

- 商户下单前缓存可选渠道编码。
- 商户接入 `charge.channel_code` 前先拉取可选项做参数校验。

注意：

- 该列表由平台统一维护。
- `default_channel_code` 表示平台当前兜底使用的默认代收渠道。
- `can_customize_channel` 表示当前商户是否有权在下单时真正覆盖平台默认代收渠道。
- 返回的是当前可用渠道，不代表你一定有权在下单时强制指定该渠道。
- 如果平台未给你的商户开通“代收自定义渠道”，即使你在 `charge.create` 传了 `channel_code`，系统也不会报错，而是忽略该值并走平台默认代收渠道。

8.3 响应示例

```
{
  "code": 1,
  "message": "ok",
  "data": {
    "can_customize_channel": false,
    "default_channel_code": "silkpays_default",
    "channels": [
      {
        "id": 11,
        "code": "silkpays_default",
        "display_name": "SilkPay Default",
        "provider": "silkpays",
        "default": true
      }
    ]
  }
}
```

8.4 响应字段说明

字段	说明
<code>data.can_customize_channel</code>	当前商户是否允许在 <code>charge.create</code> 中自定义 <code>charge.channel_code</code>
<code>data.default_channel_code</code>	平台当前默认代收渠道编码；当商户未指定渠道或无权指定时，系统回退到该值
<code>data.channels [].id</code>	平台内部渠道 ID，仅作展示
<code>data.channels [].code</code>	渠道编码；如开通自定义渠道能力，下单时传入 <code>charge.channel_code</code>
<code>data.channels [].display_name</code>	渠道展示名称
<code>data.channels [].provider</code>	渠道提供方标识
<code>data.channels [].default</code>	是否为平台当前默认代收渠道

9. 创建收款

9.1 请求路径

POST /ganymede/open/v1/charge/create

9.2 核心字段

字段	必填	说明
<code>charge.charge_amount</code>	是	收款金额，最小货币单位
<code>charge.charge_currency</code>	是	币种，如 USD
<code>charge.payment_method</code>	是	支付方式
<code>charge.channel_code</code>	否	指定代收渠道编码；仅在该商户开通代收自定义渠道生效
<code>charge.merchant_order.order_id</code>	是	商户订单号
<code>charge.merchant_order.order_items</code>	是	商品列表
<code>charge.merchant_order.order_items[].item_unit_amount</code>	是	商品单价，整数最小货币单位，不能传小数或浮点数
<code>charge.merchant_order.customer</code>	是	买家基础信息
<code>charge.extra.enable_charge_link</code>	是	是否启用收银台
<code>charge.extra.redirect_url</code>	否	本次请求级支付完成跳转未传时回退商户默认 <code>return_url</code> ；若配置了 <code>callback_host_allow</code> 其 hostname 必须命中
<code>charge.extra.notify_url</code>	否	本次请求级 Webhook 回调地址；仅在平台开启 <code>allow_notify_url_override</code> 时生效；若配置了 <code>callback_host_allow</code> 其 hostname 必须命中

9.3 条件字段规则

- 订单条目存在 `physical` 商品时，必须传 `merchant_order.shipping`。

- 订单条目存在 digital 商品时，必须传 merchant_order.billing。
- redirect_url 在 ChargeLink 模式下最终值必须存在；如果请求级 redirect_url 未传，则回退商户默认 return_url。
- notify_url 请求级覆盖仅在平台已开通 allow_notify_url_override 时生效；否则传入即按参数错误处理。
- 若商户配置了 callback_host_allowlist，则默认 notify_url / return_url 与请求级 notify_url / redirect_url 都必须命中白名单 hostname。
- callback_host_allowlist 仅支持精确 hostname；不支持 wildcard, path / query / port 不参与匹配。
- 若平台开启托管结果页，商户原始 redirect_url 仍会先按 callback_host_allowlist 校验，通过后才会改写为平台结果页地址。
- channel_code 只有在平台明确给商户开通“代收自定义渠道”时才会生效。
- 未传 channel_code，或平台未开通该能力时，系统自动走平台默认代收渠道。
- 平台当前默认代收渠道可通过 GET /ganymede/open/v1/channel/collection-list 返回的 default_channel_code 读取。
- 若最终 notify_url 为空，订单照常创建，但不会创建 Webhook 回调投递任务。
- 如果开通直连模式，通常还需要补充卡信息、风险字段与 3DS 跳转处理。

9.4 支付方式

当前文档整理到的支付方式如下，实际可用项以平台给商户开通结果为准：

payment_method	说明
flyway_checkout	收银台模式，适合首期接入
flyway_direct	直连模式，需要额外前端风控与 3DS 处理
spiral_checkout	收银台模式
v5pay_checkout	收银台模式
llooda_direct	直连模式

9.5 ChargeLink 模式

如果 `charge.extra.enable_charge_link = true`：

- `charge_link_style = redirect` 或不传时，前端跳转到返回的 `data.charge.credential.charge_link`
- `charge_link_style = embedded` 时，商户可用 `iframe` 嵌入收银台
- `embedded` 模式下，结果通过 `window.postMessage` 回传宿主页面

`embedded` 模式下的消息类型：

- `HEIGHT`
- `CHARGE_FAILED`
- `CHARGE_COMPLETED`

注意：

- 支付完成后系统会跳回 `redirect_url`
- 系统不会自动给 `redirect_url` 追加订单参数，商户需要自己在 URL 中携带上下文信息并自行校验
- 如果平台已开启托管结果页，则这里的 `redirect_url` 指的是平台结果页地址，而不是商户原始页面地址。
- 若上游要求配置可信回跳域名或 `trusted domains`，必须把 `api.speed-pay.nsngc.org`（或平台实际配置的结果页域名）加入上游白名单，否则 ChargeLink 回跳可能被拒绝。

- 上述 `trusted domains` 属于上游通道配置，不等同于商户在 Speed Pay 平台侧的 `callback_host_allowlist`。

9.6 flyway_direct 说明

`flyway_direct` 是直连卡支付模式。该模式不会先进入收银台，商户需要在服务端创建收款时提交卡信息、买家信息、账单/物流地址以及建议的浏览器设备信息。

使用前提：

- 商户必须已由平台开通直连能力；未开通时会返回参数错误。
- `payment_method` 传 `flyway_direct`。
- 卡信息统一放在 `charge.extra.flyway.card` 下。`charge.extra.card` 仅作为旧版兼容字段保留，新接入不要使用。
- `charge.merchant_order.customer.customer_client_ip` 必传，应填写真实下单买家的公网 IP。
- 建议同时传完整的 `merchant_order.billing` 与 `merchant_order.shipping`。部分上游直连通道会强校验账单地址与物流地址，即使虚拟商品也建议至少完整传 `billing`。
- `redirect_url` 用于 3DS 或上游客户动作完成后的浏览器回跳；未传时会回退商户默认 `return_url`。
- `notify_url` 用于服务端 Webhook；若平台未给商户开启请求级覆盖能力，则以商户后台默认 `notify_url` 为准。

直连卡信息字段：

字段	必填	说明
charge.extra.flyway.card.card_no	是	卡号
charge.extra.flyway.card.cvv	是	CVV
charge.extra.flyway.card.expire_year	是	过期年份，通常传 2 位年份如 28，部分通道也兼容 4 位年份如 2028
charge.extra.flyway.card.expire_month	是	过期月份，建议 2 位月份
charge.extra.flyway.card.first_name	否	持卡人名字；未传时部分通道会回退账单/买家名字
charge.extra.flyway.card.last_name	否	持卡人姓氏；未传时部分通道会回退账单/买家姓氏

直连浏览器/设备信息字段：

字段	必填	说明
charge.extra.flyway.device.ua	否	浏览器 User-Agent, 建议传
charge.extra.flyway.device.lang	否	浏览器语言, 例如 en-US
charge.extra.flyway.device.browser_lang	否	浏览器首选语言, 例如 en-US
charge.extra.flyway.device.accept	否	HTTP Accept Header
charge.extra.flyway.device.time_zone	否	买家本地时区, 例如 America/New_York
charge.extra.flyway.device.os	否	操作系统, 例如 Windows 10
charge.extra.flyway.device.browser	否	浏览器名称, 例如 Chrome
charge.extra.flyway.device.screen_height	否	屏幕高度
charge.extra.flyway.device.screen_width	否	屏幕宽度
charge.extra.flyway.device.color_depth	否	屏幕色深

直连风控辅助字段:

字段	必填	说明
charge.extra.flyway.forter_token	否	风控 token; 如商户前端已接入对应采集 SDK, 可传入
charge.extra.flyway.black_box	否	风控设备指纹; 如商户前端已接入对应采集 SDK, 可传入
charge.extra.flyway.pay_local_time	否	买家本地支付时间, 建议 ISO 8601 字符串

3DS / 客户动作处理:

- 如果响应中存在 `data.charge.credential.flyway.redirect_action`，商户前端必须跳转该地址，引导买家完成 3DS 或其他客户动作。
- 如果响应状态为 `CHARGE_CUSTOMER_ACTION_REQUIRED`，也应按同样逻辑处理跳转动作。
- 最终支付结果仍以 Webhook 或 `charge.retrieve` 查询结果为准，不要只依赖浏览器回跳判断成功。

9.7 请求示例

```
{
  "charge": {
    "charge_amount": 1024,
    "charge_currency": "USD",
    "payment_method": "flyway_checkout",
    "channel_code": "silipay_default",
    "merchant_order": {
      "order_id": "S0202603240001",
      "order_items": [
        {
          "name": "Demo Product",
          "category": "physical",
          "quantity": 1,
          "quantity_unit": "pcs",
          "item_unit_amount": 1024,
          "item_currency": "USD"
        }
      ]
    },
    "shipping": {
      "first_name": "Joe",
      "last_name": "Berge",
      "email": "buyer@example.com",
      "phone": "6362962219",
      "country_code": "US",
      "postal_code": "10036",
      "state": "NY",
      "city": "New York",
      "address_line1": "47 W 13th St",
      "address_line2": "B103"
    },
    "customer": {
      "customer_id": "CU202603240001",
      "customer_client_ip": "203.0.113.10",
      "first_name": "Joe",
      "last_name": "Berge",
      "email": "buyer@example.com",
      "country_code": "US"
    }
  }
}
```

```
    }  
  },  
  "extra": {  
    "enable_charge_link": true,  
    "charge_link_style": "redirect",  
    "redirect_url": "https://merchant.example/pay/result?  
order_no=S0202603240001",  
    "notify_url":  
"https://merchant.example/pay/webhook/orders"  
  },  
  "metadata": {  
    "order_no": "S0202603240001",  
    "utm_source": "merchant.example"  
  }  
}  
}
```

9.8 flyway_direct 请求示例

```
{
  "charge": {
    "charge_amount": 7199,
    "charge_currency": "USD",
    "payment_method": "flyway_direct",
    "merchant_order": {
      "order_id": "S0202605180001",
      "order_items": [
        {
          "name": "Mens Black Crew Tee",
          "sku": "TEE-BLK-M",
          "category": "physical",
          "description": "Classic black crew neck T-shirt with
a regular fit.",
          "url": "https://merchant.example/products/tee-blk-
m",
          "image_url": "https://merchant.example/assets/tee-
blk-m.jpg",
          "quantity": 1,
          "quantity_unit": "pcs",
          "item_unit_amount": 7199,
          "item_currency": "USD"
        }
      ],
    },
    "shipping": {
      "first_name": "Joe",
      "last_name": "Berge",
      "email": "buyer@example.com",
      "phone": "6362962219",
      "country_code": "US",
      "postal_code": "10036",
      "state": "NY",
      "city": "New York",
      "address_line1": "47 W 13th St",
      "address_line2": "B103"
    },
    "billing": {
```

```
    "first_name": "Joe",
    "last_name": "Berge",
    "email": "buyer@example.com",
    "phone": "6362962219",
    "country_code": "US",
    "postal_code": "10036",
    "state": "NY",
    "city": "New York",
    "address_line1": "47 W 13th St",
    "address_line2": "B103"
  },
  "customer": {
    "customer_id": "CU202605180001",
    "customer_client_ip": "203.0.113.10",
    "first_name": "Joe",
    "last_name": "Berge",
    "email": "buyer@example.com",
    "country_code": "US"
  }
},
"extra": {
  "enable_charge_link": false,
  "redirect_url": "https://merchant.example/pay/result?
order_no=S0202605180001",
  "notify_url":
"https://merchant.example/pay/webhook/orders",
  "flyway": {
    "card": {
      "card_no": "4111111111111111",
      "cvv": "333",
      "expire_year": "28",
      "expire_month": "12",
      "first_name": "Joe",
      "last_name": "Berge"
    },
    "device": {
      "ua": "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/124.0.0.0
Safari/537.36",
```



```
    "lang": "en-US",
    "browser_lang": "en-US",
    "accept":
"text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0
.8",
    "time_zone": "America/New_York",
    "os": "Windows 10",
    "browser": "Chrome",
    "screen_height": "1080",
    "screen_width": "1920",
    "color_depth": "24"
  }
},
"metadata": {
  "order_no": "S0202605180001",
  "utm_source": "merchant.example"
}
}
```

9.9 响应重点字段

字段	说明
<code>data.charge.charge_id</code>	支付单号，后续查询和退款都要用到
<code>data.charge.charge_status</code>	支付状态
<code>data.charge.credential.charge_link</code>	收银台跳转地址
<code>data.charge.credential.flyway.redirect_action</code>	直连模式下的 3DS 跳转地址
<code>data.charge.credential.flyway.redirect_method</code>	直连模式下的跳转方法，通常为 GET
<code>data.charge.credential.flyway.redirect_action_type</code>	直连模式下的跳转动作类型，通常为 <code>redirect</code>
<code>data.charge.transaction_no</code>	上游渠道交易号
<code>data.charge.refundable_amount</code>	当前剩余可退款金额

9.10 收款状态

状态	说明
<code>INITIALIZED</code>	已初始化
<code>CREATE_FAILED</code>	创建失败
<code>CREATED</code>	已创建，待用户完成支付
<code>CHARGE_CUSTOMER_ACTION_REQUIRED</code>	需要买家继续完成 3DS 或其他客户动作
<code>CHARGE_FAILED</code>	支付失败
<code>CHARGE_CLOSED</code>	收款关闭
<code>CHARGE_COMPLETED</code>	支付成功

9.11 结果判定说明

- 顶层 `code=1` 、 `message=ok` 表示接口请求已被成功受理，不代表收款创建成功或支付成功。
- 商户应始终以 `data.charge.charge_status` 作为业务结果判断依据。
- 当 `data.charge.charge_status=CREATE_FAILED` 时，表示收款创建失败，应结合 `data.charge.failure_code` 与 `data.charge.failure_message` 继续排查。
- 当 `data.charge.charge_status=CHARGE_CUSTOMER_ACTION_REQUIRED` 或响应中存在 `data.charge.credential.flyway.redirect_action` 时，表示直连支付需要买家继续完成 3DS / 客户动作，商户前端应跳转该地址。
- 仅当返回了可用的支付凭据（如 `data.charge.credential.charge_link` 或直连场景下的跳转动作）后，才应继续引导用户完成支付。

10. 查询收款

10.1 请求路径

`POST /ganymede/open/v1/charge/retrieve`

10.2 请求体

```
{
  "charge_id": "ch_demo_001"
}
```

10.3 使用建议

- 用户支付完成后前端先返回，但商户还未收到 Webhook
- 商户做定时补单
- 商户做日终对账

查询收款返回的核心对象与 `charge.create` 中的 `data.charge` 一致。

11. 创建退款

11.1 请求路径

POST /ganymede/open/v1/refund/create

11.2 核心字段

字段	必填	说明
refund.charge_id	是	原支付单号
refund.refund_amount	是	退款金额
refund.refund_currency	是	退款币种，应与支付币种一致
refund.metadata	否	商户退款附加信息，建议传 JSON 字符串

11.3 请求示例

```
{
  "refund": {
    "charge_id": "ch_demo_001",
    "refund_amount": 310,
    "refund_currency": "USD",
    "extra": {
      "reason": "customer_request"
    },
    "metadata": "{\"refund_no\":\"RF202603240001\"}"
  }
}
```

11.4 退款状态

状态	说明
INITIALIZED	已初始化
CREATE_FAILED	创建失败
CREATED	已创建，待上游处理
REFUND_FAILED	退款失败
REFUND_COMPLETED	退款成功

12. 批量查询退款

12.1 请求路径

POST /ganymede/open/v1/refund/batch-retrieve

12.2 请求体

```
{
  "refund_ids": [
    "re_demo_001",
    "re_demo_002"
  ]
}
```

12.3 使用建议

- 批量对账
- 退款回调延迟时补查
- 客服系统集中刷新退款状态

13. 查询争议

13.1 请求路径

POST /ganymede/open/v1/dispute/retrieve

13.2 请求体

```
{
  "dispute_id": "du_demo_001"
}
```

13.3 争议状态

状态	说明
OPEN	争议创建
NEED_ACTION	待补充材料
UNDER_REVIEW	审核中
LOST	争议败诉
WON	争议胜诉

14. Webhook

14.1 事件列表

事件	说明
charge.succeeded	收款成功
charge.failed	收款失败
refund.succeeded	退款成功
refund.failed	退款失败
dispute.opened	争议创建
dispute.lost	争议败诉
dispute.won	争议胜诉

14.2 回调体示例

```
{
  "event_id": "evt_WBPAuVba35Ne6DB9H",
  "event_type": "charge.succeeded",
  "event_born_ts": 1753842653,
  "object_id": "ch_WBPAu2NSWR1b5NAqe",
  "object_type": "charge",
  "nonce": "b742ca01-d455-45a1-b795-f2152e1dccd0"
}
```

14.3 回调投递目标

Webhook 最终投递地址按以下顺序决策：

1. 订单创建时解析出的最终 `notify_url`
2. 若订单未保存单独回调地址，则回退商户默认 `notify_url`

补充说明：

- 如果商户默认 `notify_url` 为空，且该笔订单也没有请求级 `notify_url`，系统只会记录事件，不会创建回调投递。
- 只有存在最终 `notify_url` 的事件，才会进入 ACK 与重试逻辑。
- 商户若要自行维护默认 `notify_url` / `return_url`，前提是平台为该商户开启了 `allow_callback_profile_self_manage`。

14.4 验签规则

签名头：

```
X-BF-Signature
```

签名算法：

- HmacSHA512
- Key 为 Base64 解码后的 `merchant_webhook_secret`

待签名串生成规则：

1. 取回调 JSON 中所有顶层字段。
2. 按字段名升序排序。
3. 按 `key=value` 形式使用 `&` 拼接。

模板如下：

```
event_born_ts=${event_born_ts}&event_id=${event_id}&event_type=${event_type}&nonce=${nonce}&object_id=${object_id}&object_type=${object_type}
```

14.5 正确 ACK

Webhook 处理成功后必须返回：

- HTTP 状态码：200
- Body：

```
{"acknowledged":true}
```

14.6 重试策略

若未返回正确 ACK，系统会在 1 小时内最多重试 9 次。

重试间隔如下：

4s, 8s, 16s, 32s, 64s, 128s, 256s, 512s, 1024s

14.7 Webhook 处理建议

- 对 event_id 做幂等去重
- 验签失败直接拒绝处理
- 先落库，再执行业务逻辑
- 返回 ACK 后再做耗时任务，避免无意义重试

15. 常见错误码

15.1 预检错误码

错误码	含义
0	系统内部错误
2	参数错误
3	请求体格式错误
4	存在并发中的幂等请求
5	认证失败
6	请求被限流
1001	缺少必填参数
1002	参数非法
1013	无可支付网络
1014	支付单或争议对象不存在
1016	开启 ChargeLink 但未传 redirect_url
1019	callback_data 超长
5002	可退款金额不足
5003	退款币种不一致
5004	只有成功支付单才能退款
5005	超出退款时间窗口

15.2 业务失败码

业务对象进入稳定失败状态后，可进一步关注对象上的失败码：

错误码	含义
11000	支付网络错误
11001	支付网络创建失败
11006	高风险卡
11014	余额不足
11015	3DS 验证失败
11018	发卡行拒绝交易
50000	收款过期
70000	欺诈预警争议
70004	持卡人争议
70005	退款问题争议

16. 沙箱联调

16.1 测试说明

- 以下测试卡仅用于 SANDBOX 商户联调
- SANDBOX 商户使用的仍是统一 OpenAPI 域名，不需要切换独立沙箱域名
- SANDBOX 商户下单、退款、Webhook、对账事件都会按正式链路落库，但不会调用真实上游
- 若已配置可访问的 `notify_url`，沙箱也会按相同重试规则投递模拟事件；若最终 `notify_url` 为空，则不会发起投递
- 除特别说明外，过期时间需晚于当前月份
- 除特别说明外，CVV 可填任意三位数字

16.2 测试卡

支付方式	卡号	过期时间	CVV	说明
flyway_checkout / flyway_direct	4111111111111111	晚于当前月份	任意三位	无 3DS
flyway_checkout / flyway_direct	4000000000001091	晚于当前月份	任意三位	无 3DS
flyway_checkout / flyway_direct	4242424242424242	晚于当前月份	任意三位	有 3DS, 密码 Checkout1!
flyway_checkout / flyway_direct	4066330000000004	晚于当前月份	任意三位	有 3DS, 密码 Checkout1!
spiral_checkout	4000027891380961	01/29	123	无 3DS, 持卡人 John Smith
spiral_checkout	2221008123677736	01/30	151	金额超过 USD 151 时触发 3DS, 持卡人 CL-BRW 2

17. 对接建议

- 首期推荐优先接入 flyway_checkout + redirect
- 把 Webhook 作为订单最终状态源
- 把 charge_id、refund_id、dispute_id、event_id 全量落库
- 所有失败响应都要完整记录原始 code、message 与对象失败码
- 如平台给你开通了请求级 notify_url 覆盖能力, 建议仅用于极少数特殊订单, 常规订单仍优先使用商户默认回调配置
- 正式切到 LIVE 商户前, 先在 SANDBOX 商户完成重试、幂等、退款、争议查询链路验证

18. 版本记录

版本	日期	说明
v1.4	2026-05-18	补齐 flyway_direct 直连模式字段说明、卡信息路径、设备风控字段、3DS 跳转处理与直连请求示例
v1.3	2026-04-01	增加 callback_host_allowlist 说明，明确默认/请求级回调域名校验、平台托管结果页 rewrite 与 trusted domains 区分
v1.2	2026-03-27	增加默认回调配置、请求级 notify_url 覆盖、OpenAPI IP 白名单与“无最终 notify_url 不投递 callback”说明
v1.1	2026-03-26	调整为单部署商户级沙箱模式，取消独立沙箱域名表述
v1.0	2026-03-24	首版单册商户接入手册，覆盖当前已实现 OpenAPI 最小接口集